

Plataforma .NET

Mensajería, persistencia y despliegue

Arquitectura del Software

Curso 2025/26

RabbitMQ en .NET

RabbitMQ

- Broker de mensajes (intermediario). Conceptos fundamentales:
 - Productor: publica mensajes.
 - Exchange: recibe y enruta los mensajes.
 - Cola: almacena los mensajes hasta que se consumen.
 - Binding: regla que conecta un Exchange con una Cola.
 - Consumidor: lee y procesa mensajes de la cola.

RabbitMQ: Tipos de Exchange

- Direct: El mensaje va a la cola cuya routing key coincide exactamente con la del mensaje.
- Fanout: El mensaje se envía a todas las colas enlazadas (broadcast).
- Topic: Coincidencia por patrones con wildcards.
- Headers: Enruta por cabeceras del mensaje.
- Default: Exchange sin nombre. Se usa como routing key el nombre de la cola.

Usar RabbitMQ en .NET

Para poder trabajar con RabbitMQ en un proyecto .NET, debemos añadir la dependencia:

```
dotnet add package RabbitMQ.Client
```

[Documentación del API](#)

Conectar a un nodo RabbitMQ

- Para abrir una conexión se instancia `ConnectionFactory` configurándola con los datos del host. El método de apertura de conexión es asíncrono.

```
ConnectionFactory factory = new ConnectionFactory();  
  
factory.UserName = user;  
factory.Password = pass;  
factory.VirtualHost = vhost;  
factory.HostName = hostName;  
  
IConnection conn = await factory.CreateConnectionAsync();
```

Conectar a un nodo RabbitMQ

- También podemos inicializarla con una Uri:

```
ConnectionFactory factory = new ConnectionFactory();  
factory.Uri = new Uri("amqp://user:pass@hostName:port/vhost");  
IConnection conn = await factory.CreateConnectionAsync();
```

- La interfaz IConnection se utiliza para abrir el canal:

```
IChannel channel = await conn.CreateChannelAsync();
```

- El canal lo usamos para publicar, consumir, declarar colas, etc.

Desconectar de RabbitMQ

Nos podemos desconectar del nodo cerrando el canal y la conexión:

```
await channel.CloseAsync(); //cierra  
await conn.CloseAsync();  
await channel.DisposeAsync(); // libera recursos  
await conn.DisposeAsync();
```

Declaración de Exchanges y Colas

- Los exchanges y las colas se pueden declarar desde código y vincularlos:

```
await channel.ExchangeDeclareAsync(  
    exchange: "mi-exchange",  
    type: ExchangeType.Direct, // Direct, Fanout, Topic, Headers  
    durable: true,             // sobrevive reinicios del broker  
    autoDelete: false,         // a true se borra cuando no quedan bindings  
    arguments: null);  
await channel.QueueDeclareAsync(  
    queue: "mi-cola",  
    durable: true,             // sobrevive reinicios  
    exclusive: false,          // a true solo accesible por esta conexión  
    autoDelete: false,         // a true se borra cuando no quedan consumers  
    arguments: null);  
await channel.QueueBindAsync(  
    queue: "mi-cola", exchange: "mi-exchange",  
    routingKey: "mi-routing-key", arguments: null  
);
```

Productor de mensajes

- Para publicar mensajes en un exchange se usa el método

`BasicPublishAsync` :

```
byte[] body = System.Text.Encoding.UTF8.GetBytes("¡Hola Mundo!");  
var props = new BasicProperties();  
  
await channel.BasicPublishAsync(  
    exchange: "exchangeName",  
    routingKey: "arso",  
    mandatory: false,  
    basicProperties: props,  
    body: body);
```

Productor de mensajes

- `BasicProperties` nos permiten definir metadatos del mensaje.

```
byte[] messageBodyBytes = System.Text.Encoding.UTF8.GetBytes("¡Hola Mundo!");  
  
var props = new BasicProperties();  
props.ContentType = "text/plain";  
props.DeliveryMode = 2;  
  
await channel.BasicPublishAsync(exchangeName, routingKey,  
    mandatory: true, basicProperties: props, body: messageBodyBytes);
```

Consumidor de mensajes

- La forma recomendada de recibir mensajes es estableciendo una suscripción utilizando la interfaz `IAsyncBasicConsumer`.

```
var consumer = new AsyncEventingBasicConsumer(channel);
consumer.Received += async (ch, ea) =>
{
    var body = ea.Body.ToArray();
    var message = Encoding.UTF8.GetString(body);
    Console.WriteLine($"Mensaje recibido: {message}");
    await channel.BasicAckAsync(ea.DeliveryTag, false);
};
string consumerTag = await channel.BasicConsumeAsync(queueName, false,
consumer);
```

Consumidor de mensajes

En el código anterior:

- Se crea el consumidor ligado al canal.
- `Received` es un evento que se dispara cada vez que llega un mensaje.
- Con `+=` la función se suscribe al evento. Sus parámetros son `ch` (el canal) y `ea` (todo sobre el mensaje recibido)
- `BasicAckAsync` : confirma el mensaje.
- `BasicConsumeAsync` : inicia el consumo de mensajes. El segundo argumento `autoAck` a `false` indica confirmación manual.

Microservicios .NET: Publicar un evento

- Necesitamos el evento a enviar

```
public abstract class IntegrationEvent
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public DateTime OccurredAt { get; init; } = DateTime.UtcNow;
    public abstract string RoutingKey { get; }
}

public class PagoConfirmado : IntegrationEvent {
    public override string RoutingKey => "pago.confirmado";
    public int PedidoId { get; set; }
    public string ReferenciaTransaccion { get; set; }
}
```

- No es necesario tener la clase abstracta.
- En lugar de `class` se puede usar `record` también.

Microservicio .NET: Publicar un evento

- Usamos una interfaz para desacoplar el servicio de cualquier implementación concreta de publicación.

```
public interface IEventPublisher
{
    Task PublishAsync(IntegrationEvent evento);
}
```

- Si no tenemos la clase base del evento, se puede dejar parametrizado

```
public interface IEventPublisher
{
    Task PublishAsync<T>(T evento);
}
```

Microservicio .NET: Publicar un evento

```
public class RabbitMqEventPublisher : IEventPublisher
{
    public RabbitMqEventPublisher(IConnection connection, IConfiguration config)
    {
        // init _connection y _exchangeName ...
    }
    public async Task PublishAsync(IntegrationEvent evento)
    {
        await using var channel = await _connection.CreateChannelAsync();
        await channel.ExchangeDeclareAsync(exchange: _exchangeName,
            type: ExchangeType.Topic, durable: true);
        var routingKey = evento.RoutingKey;

        var json = JsonSerializer.Serialize(evento);
        var body = Encoding.UTF8.GetBytes(json);

        await channel.BasicPublishAsync(exchange: _exchangeName,
            routingKey: routingKey, mandatory: false,
            basicProperties: null, body: body);
    }
}
```

Microservicio .NET: Publicar un evento

- Del código anterior destacamos:
 - `await using var channel` : cuando el bloque termine, llamará automáticamente a `channel.DisposeAsync()` para cerrar y liberar el canal.
 - `JsonSerializer.Serialize` y `Encoding.UTF8.GetBytes` : serializa el evento a JSON en formato `byte[]` .
 - `mandatory: false` : descarta el mensaje si no hay destino. Evitamos lanzar error.

Microservicio .NET: Publicar un evento

- En Program.cs incluimos lo necesario en el contenedor de inyección de dependencia.

```
var factory = new ConnectionFactory
{
    Uri = new Uri(builder.Configuration["RabbitMQ:ConnectionString"]!)
};
var connection = await factory.CreateConnectionAsync();
builder.Services.AddSingleton<IConnection>(connection);
builder.Services.AddSingleton<IEventPublisher, RabbitMqEventPublisher>();
```

- En appsettings.json

```
"RabbitMQ": {
  "ConnectionString": "amqps://user:pass@cluster/vhost",
}
```

Microservicio .NET: Consumir un evento

- Una manera de implementar un consumidor es como servicio en segundo plano.
- Creamos una clase que herede de `BackgroundService` e implemente `ExecuteAsync`.

```
public class RabbitMqPagoConsumer : BackgroundService
{
    private readonly IConnection _connection;
    private IChannel? _channel;
    //..
}
```

Microservicio .NET: Consumir un evento

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    _channel = await _connection.CreateChannelAsync();

    // declarar el exchange, la cola y hacer el binding

    var consumer = new AsyncEventingBasicConsumer(_channel);
    consumer.ReceivedAsync += async (_, ea) =>
    {
        var body = ea.Body.ToArray();
        var json = Encoding.UTF8.GetString(body);
        var evento = JsonSerializer.Deserialize<PagoConfirmado>(json);
        Console.WriteLine($"Pago recibido: {evento?.ReferenciaTransaccion}");
        await _channel.BasicAckAsync(ea.DeliveryTag, multiple: false);
    };
    //..
}
```

Microservicio .NET: Consumir un evento

Del código anterior destaca:

- En el callback que se ejecuta por cada mensaje que llega:
 - `ea.Body.ToArray()` : extrae los bytes del mensaje.
 - `Encoding.UTF8.GetString` : convierte los bytes a string JSON.
 - `JsonSerializer.Deserialize<PagoConfirmado>` : deserializa al tipo concreto.

Microservicio .NET: Consumir un evento

Para terminar:

```
await _channel.BasicConsumeAsync(  
    queue: Queue,  
    autoAck: false,  
    consumer: consumer  
);  
await Task.Delay(Timeout.Infinite, stoppingToken);
```

- `_channel.BasicConsumeAsync` : Arranca el consumo.
- `Task.Delay` : Mantiene el `ExecuteAsync` vivo indefinidamente.

Microservicio .NET: Consumir un evento

```
public override async Task StopAsync(CancellationToken cancellationToken)
{
    if (_channel is not null)
    {
        await _channel.CloseAsync();
        await _channel.DisposeAsync();
    }
    await base.StopAsync(cancellationToken);
}
```

- Cuando la app se para, cierra `_channel` limpiamente antes de que .NET destruya el servicio. `base.StopAsync` desbloquea el `Task.Delay` definido anteriormente.

ORM en .NET: Entity Framework Core

Entity Framework Core

- Entity Framework (EF) Core es un ORM para .NET
 - Permite trabajar con una base de datos usando objetos .NET.
 - Reduce el código de acceso a datos necesario.
- Da soporte a muchos motores de bases de datos [\[ver\]](#).
- El acceso se realiza por medio de un modelo. El modelo lo forman las clases de entidad y un objeto de contexto que representa una sesión con la base de datos.
- [Documentación.](#)

Incluir EF Core en un proyecto .NET

- Hay que instalar el paquete adecuado para el proveedor de base de datos que se quiere utilizar.
- Por ejemplo, para MySQL:

```
dotnet add package Pomelo.EntityFrameworkCore.MySql
```

Flujos de trabajo principales

- Dos enfoques:
 - Code First:
 - Defines las clases C# (entidades).
 - EF Core genera la base de datos con migraciones.
 - Database First:
 - Partimos de una base de datos existente.
 - EF Core genera las clases a partir del esquema.

DbContext

- Una instancia de `DbContext` representa una sesión con la base de datos y se puede usar para consultar y guardar instancias de las entidades.
- Gestiona:
 - La conexión a la base de datos.
 - El seguimiento de los cambios.
 - Las operaciones CRUD.
 - Las transacciones.

DbContext

```
public class BloggingContext : DbContext // Hereda de DbContext
{
    public DbSet<Blog> Blogs { get; set; } // Representa una tabla
    public DbSet<Post> Posts { get; set; }

    // OnConfiguring para configurar el proveedor, cadena de conexión...
    protected override void OnConfiguring(DbContextOptionsBuilder
        optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            @"Server=(localdb)\mssqllocaldb;
            Database=Blogging;Trusted_Connection=True;
            ConnectRetryCount=0");
    }
}
```

DbContext en la inserción de dependencias

- EF Core se puede agregar al contenedor de inyección de dependencias mediante `AddDbContext` en `Program.cs`.
- En aplicaciones ASP.NET Core tiene sentido vincular la vida útil del contexto a la de una solicitud HTTP.

```
var connectionString =  
    builder.Configuration.GetConnectionString("DefaultConnection")  
    ?? throw new InvalidOperationException("Connection string"  
    + "'DefaultConnection' not found.");  
  
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlServer(connectionString));
```

- `ApplicationDbContext` es una subclase de `DbContext`.

DbContext en la inserción de dependencias

- `ApplicationDbContext` debe exponer un constructor público con un parámetro `DbContextOptions<ApplicationDbContext>`.

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
}
```

- De esta manera, `ApplicationDbContext` se puede utilizar por inyección en el constructor.

Modelo en EF Core

- EF Core usa un modelo de metadatos para describir como los tipos de entidad de la aplicación se mapean a la base de datos.
- El modelo se puede construir usando convenciones.
 - Cada clase -> una tabla.
 - Cada propiedad -> una columna.
 - La propiedad con nombre `Id` o `<NombreClase>Id` -> clave primaria.

Configurando el Modelo

- El modelo se puede customizar haciendo uso de anotaciones o llamando a los métodos de `ModelBuilder` también conocido como *fluent API* en el método `OnModelCreating`.
- El uso de anotaciones es más rápido y autodescriptivo. Configuración acoplada al modelo.
- Fluent API centraliza la configuración, es más potente y separa la configuración del modelo.
- Se pueden combinar ambas. [Ver documentación.](#)

Anotaciones: Propiedades.

```
[Table("productos")]           // Nombre de tabla personalizado
public class Producto
{
    [Key]                        // Clave primaria explícita
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]                  // NOT NULL
    [MaxLength(200)]            // VARCHAR(200)
    public string Nombre { get; set; }

    [Column("precio_venta")]    // Nombre de columna personalizado
    [Range(0, 99999)]
    public decimal Precio { get; set; }

    [NotMapped]                 // No persiste en BD
    public string NombreFormateado => $"[{Id}] {Nombre}";
}
```

Fluent API

```
public class TiendaContext : DbContext
{
    public DbSet<Producto> Productos { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Producto>(entity =>
        {
            entity.ToTable("productos");
            entity.HasKey(p => p.Id);
            entity.Property(p => p.Nombre)
                .IsRequired()
                .HasMaxLength(200);
            entity.Property(p => p.Precio)
                .HasColumnType("decimal(10,2)");
        });
    }
}
```

Mapeo de Asociaciones

- Para poder mapear las asociaciones del modelo relacional en EF Core se necesita:
 - Añadir una clave primaria a cada tipo de entidad.
 - Añadir una clave ajena a un tipo de entidad.
 - Asociar la referencia entre tipos de entidad con la clave primaria y la clave ajena para formar una única configuración.

[Ver configuración de asociaciones.](#)

Asociación Uno a Muchos

```
public class Categoria
{
    public int Id { get; set; }
    public string Nombre { get; set; }
    public ICollection<Producto> Productos { get; set; } = new List<Producto>();
}
public class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; }
    public int CategoriaId { get; set; }
    public Categoria Categoria { get; set; }
}
```

Asociación Uno a Muchos

```
// Fluent API
modelBuilder.Entity<Producto>()
    .HasOne(p => p.Categoria)
    .WithMany(c => c.Productos)
    .HasForeignKey(p => p.CategoriaId);
// O también
modelBuilder.Entity<Categoria>()
    .HasMany(c => c.Productos)
    .WithOne(p => p.Categoria)
    .HasForeignKey(p => p.CategoriaId);
// Con shadow foreign key
modelBuilder.Entity<Categoria>()
    .HasMany(c => c.Productos)
    .WithOne(p => p.Categoria)
    .HasForeignKey("CategoriaId");
```

Agregados en EF Core

- Patrón de Domain-Driven Design (DDD)
- Un **agregado** es un grupo de entidades que forman una unidad de consistencia.
- La raíz del agregado es el punto de entrada al agregado.
- El `DbSet` del contexto expone solamente la raíz.

Owned Entity Types en EF Core

- Se refiere a entidades que son parte de otra entidad propietaria (*owner*) y no puede existir sin ella.
- Conceptualmente similares a los agregados.

```
[Owned]
public class StreetAddress
{
    public string Street { get; set; }
    public string City { get; set; }
}
public class Order
{
    public int Id { get; set; }
    public StreetAddress ShippingAddress { get; set; }
}
```

Owned Entity Types en EF Core

```
modelBuilder.Entity<Order>().OwnsOne(p => p.ShippingAddress);
```

- En el caso de colecciones

```
public class Distributor
{
    public int Id { get; set; }
    public ICollection<StreetAddress> ShippingCenters { get; set; }
}
// Fluent API
modelBuilder.Entity<Distributor>().OwnsMany(
    p => p.ShippingCenters, a =>
    {
        a.WithOwner().HasForeignKey("OwnerId");
        a.Property<int>("Id");
        a.HasKey("Id");
    });
```

Operaciones CRUD básicas

```
using var context = new TiendaContext();

// CREAR (CREATE)
var producto = new Producto { Nombre = "Teclado", Precio = 49.99m };
context.Productos.Add(producto); // DbSet<Producto> Productos
await context.SaveChangesAsync();
// LEER (READ)
var productos = await context.Productos
    .Where(p => p.Precio < 100)
    .Include(p => p.Categoria)      // Eager loading de la relación
    .ToListAsync();
// ACTUALIZAR (UPDATE)
var p = await context.Productos.FindAsync(1);
p.Precio = 59.99m;
await context.SaveChangesAsync(); // Change Tracker detecta el cambio
// también existe contexto.Productos.Update(producto)
// BORRAR (DELETE)
context.Productos.Remove(p);
await context.SaveChangesAsync();
```

Creación de la base de datos y migración

- En desarrollo podemos crear y destruir la base de datos continuamente. En producción lo ideal es activar las migraciones.

```
// crear destruir
db.Database.EnsureDeleted();
db.Database.EnsureCreated();
// activar migración
db.Database.Migrate();
```

Las migraciones requieren instalar [EF Core tools]
(<https://learn.microsoft.com/en-us/ef/core/cli/dotnet>).

Creamos una nueva migración cada vez que se modifica el modelo.

```
dotnet ef migrations add NombreDescriptivo
```

Despliegue en Docker

Despliegue servicio .NET en Docker

- Necesitamos crear un archivo **Dockerfile** en el directorio raíz de nuestro servicio.
- Creamos un fichero dockerfile separado en distintas etapas para obtener una imagen de un tamaño más reducido. La idea es:
 - Usamos un SDK con todo lo necesario para compilar y publicar la aplicación a un directorio
 - Usamos una imagen base pequeña que solo tenga el runtime y copiamos la carpeta de publicación de la etapa anterior para generar una imagen final ligera.

Despliegue servicio .NET en Docker

- En el **Dockerfile** de esta presentación, se definen las siguientes etapas:
 - **base** : se define la base de la imagen final dónde se ejecutará la aplicación.
 - **build** : etapa de restaurar paquetes y compilar. Se usa un SDK completo.
 - **publish** : publicar el proyecto y sus dependencias.
 - **final** : se genera la imagen final.

Dockerfile

```
# 1. Etapa base
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 8080
```

- Usamos la imagen oficial de runtime de .NET 8
- Creamos el directorio de trabajo dentro del contenedor
- Indicamos el puerto en el que escuchará la aplicación

Dockerfile

```
# 2. Build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY *.csproj ./
RUN dotnet restore

COPY . ./
RUN dotnet build -c Release -o /app/build
```

- Usamos el SDK completo para compilar y creamos la carpeta de trabajo para el código fuente
- Hacemos el `restore` (restaurar dependencias) con el archivo `.csproj`.
- Copia todo el código fuente y compila en modo `Release`.

Dockerfile

```
# 3. Publish  
FROM build AS publish  
RUN dotnet publish -c Release -o /app/publish
```

- Partiendo de la etapa anterior, la renombra a `publish`
- Genera los archivos finales listos para ejecutar

Dockerfile

```
# 4. Final
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "NombreAPI.dll"]
```

- Parte de la imagen base más ligera
- Se mueve al directorio `/app`
- Copia los binarios compilados con el SDK completo en la etapa anterior al directorio de ejecución.
- Define el comando a ejecutar cuando se inicie el contenedor. Debe coincidir con el nombre del proyecto.

docker-compose

En `docker-compose` podemos definir las variables de entorno para que reescriban lo indicado en `appsettings.json`. Levantamos todo con `docker compose up --build`

```
services:
  bookle-api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    environment:
      - MongoDB__Uri=mongodb://mongodb:27017
      - MongoDB__Database=arso2026
      - MongoDB__Collection=actividades.net
    depends_on:
      mongodb:
```